Testing Java Atomic Classes

Ahsanul Kabir

Half Semester Project
CIS 613 - W26

Outline

1. Premise
2. Test Execution
3. Results
   - AtomicBoolean Test Suite
   - AtomicInteger Test Suite
4. Metrics
5. Model Checking
6. Discussion

Motivation & Goal

- **Goal**: Write tests for part of the Java utility library
- **Scope**: *AtomicBoolean* and *AtomicInteger* (from *java.util.concurrent.atomic*)
- **Driving question**: Why does Java define separate atomic classes when Boolean, Integer, and primitives already exist?
- **Approach**: Test base and primitive types first to establish the need, then test atomic classes for functionality and atomicity

Project Setup

- Language: Java 17
- Build tool: Maven
- Test framework: JUnit 5
- Repo: github.com/kowshid/cis-613-project
- Kept only eager-evaluation methods, lazy evaluation excluded

Test Plan

1. Base class & primitive behavior under multi-threaded scenarios → establish the need for atomic classes
2. Functionality of *AtomicBoolean* and *AtomicInteger* → verify methods work correctly
3. Atomicity → verify thread-safety under heavy concurrency

Test Generation Strategy

- *AtomicBoolean*: Equivalence Class Testing (input domain is just true/false)
- *AtomicInteger*: Robust Boundary Value Testing using values: $MIN_VALUE - 1, MIN_VALUE, -1, 0, 1, MAX_VALUE, MAX_VALUE + 1$ along with Normal Boundary Value, Special Case, and Equivalence Class Testing for different methods
- Concurrency tests: 999 threads, repeated 20 times, randomly chosen numbers

AtomicBoolean Test Suite

- 30 test methods total
- Coverage:
  - Constructors (3 tests)
  - compareAndSet (4 tests — equivalence classes)
  - set / getAndSet (4 tests)
  - toString (2 tests)
  - compareAndExchange, compareAndExchangeAcquire, compareAndExchangeRelease (12 tests)
  - Atomicity tests (5 tests, each repeated $20x$)

AtomicInteger Test Suite

- 123 test methods total
- Coverage:
  - Constructors & set / getAndSet (22 tests — Robust BV)
  - compareAndSet (8 tests)
  - getAndIncrement / getAndDecrement / getAndAdd (17 tests)
  - incrementAndGet / decrementAndGet / addAndGet (17 tests)
  - compareAndExchange family (24 tests)
  - getAndUpdate / updateAndGet / getAndAccumulate / accumulateAndGet (20 tests — Special Case)
  - Atomicity tests (15 tests, each repeated 20x)

AtomicBoolean: Stale Condition Test Results

Test Setup:

1. One reader thread + one writer thread sharing a boolean
2. Writer flips false → true after burning CPU cycles
3. Reader spins until it sees true (5-second timeout)

AtomicBoolean: Stale Condition Test Results

Test Setup:

1. One reader thread + one writer thread sharing a boolean
2. Writer flips false → true after burning CPU cycles
3. Reader spins until it sees true (5-second timeout)

Results:

- *Boolean*: reader stuck, never saw the update
- *boolean*: reader stuck, never saw the update
- *AtomicBoolean*: reader exited successfully

AtomicBoolean: Functionality & Atomicity Results

- **Functionality**: All 25 functionality tests passed using equivalence class testing
- **Atomicity**: All 5 atomicity tests passed across 20 repetitions each
  - 999 threads race to flip false $\rightarrow$ true
  - Exactly 1 thread observes the initial false state every time
  - Confirms no duplicate *once-only* execution

AtomicInteger: Race Condition Test Results

Test Setup:

1. 999 threads, each performs 1,000 increments on a shared counter
2. Expected: 999 x 1,000 = 999,000

AtomicInteger: Race Condition Test Results

Test Setup:

1. 999 threads, each performs 1,000 increments on a shared counter
2. Expected: 999 x 1,000 = 999,000

Results:

- *Integer*: Outputs $\sim$ 125,075 — massive lost updates
- *int*: Outputs $\sim$ 996,616 — still loses updates
- *AtomicInteger*: Actual = 999,000 — every update preserved

AtomicInteger: Functionality & Atomicity Results

Functionality:

- All in-range tests passed
- Key finding: *AtomicInteger* does not prevent integer overflow
  - $MAX_VALUE + 1$ silently wraps to a negative number
  - $MIN_VALUE - 1$ silently wraps to a positive number
  - IDE shows a warning, but compiler does not throw an error
- Overflow safety is the user's responsibility

AtomicInteger: Functionality & Atomicity Results

Functionality:

- All in-range tests passed
- Key finding: *AtomicInteger* does not prevent integer overflow
  - $MAX_VALUE + 1$ silently wraps to a negative number
  - $MIN_VALUE - 1$ silently wraps to a positive number
  - IDE shows a warning, but compiler does not throw an error
- Overflow safety is the user's responsibility

Atomicity:

- All 15 atomicity tests passed across 20 repetitions
- Pattern: Initialize to 0, observe how many threads see 0 before update to 1
- Result: Exactly 1 thread observes the initial value in every method tested

Test Coverage

Class	Method	Line	Branch
AtomicBoolean	100% (11/11)	95% (22/23)	100% (2/2)
AtomicInteger	100% (21/21)	100% (48/48)	66% (16/24)
Overall	**100%**	**98%**	**69%**

Mutation Testing

- Overall mutation score: 98%
- 58 mutations generated, 57 killed
- Survivor: condition negation in *accumulateAndGet* (out of scope)
- Run time: $\sim$ 8 minutes 16 seconds

Model Checking

- **Tool**: Java Path Finder
- **Why**: Concurrency tests are probabilistic — a single passing run isn't proof. JPF explores all thread interleavings.
- **Constraints**:
  - Java 11 required for JPF compatibility
  - Reduced thread count from 999 $\rightarrow$ 3 for memory limits
  - Tested the JDK class directly, not my copy
- **Results**: No errors detected for both AtomicBoolean and AtomicInteger
  Confirms thread-safety holds across all explored interleavings

Lessons Learned

1. Java's *concurrent* library is well-built — the atomic classes deliver on their thread-safety contract
2. Race conditions are **probabilistic** — I had to deliberately stress my code (timing, sleeps, cycle-burning) to make race conditions reliably appear. This is why concurrency bugs are notoriously hard to find in real code.
3. Overflow safety is delegated to the user — atomic classes guarantee atomicity, not numeric correctness. All variables eventually live in fixed-bit memory; the JVM cannot police every value.

Thank You.

Any Questions?